

CLI: main_chunking

Dokumente chunken, Strategien vergleichen, Kennzahlen lesen

Zweck

main_chunking.py ist die Bedienungsschicht über der Bibliothek rag_project.pipeline.chunking. Die Bibliothek enthält nur das Teilen und Kennzahlen, die Anwendung die Kommandozeile, die Ausgabe, das Dateischreiben und die Exit-Codes.

Der Einstieg fährt die vollständige Vorverarbeitung: laden, bereinigen, chunken. Wer nur eine Stufe sehen will, nimmt die eigenen Einstiege loading und cleaning.

Aufruf

- `uv run chunking` — vollständiger Lauf mit den Standardwerten
- `uv run chunking docs/handbuch.pdf --strategy recursive --chunk-size 800`
- `uv run chunking --compare` — alle Strategien bei gleichem Zielwert
- `uv run chunking --handbook --show 5` — Handbuch-Trenner, fünf Chunks zeigen
- `uv run python -m rag_project.app.main_chunking --help` — Übersicht der Optionen

Der Ablauf

Ein Lauf besteht aus drei Schritten, die in dieser Reihenfolge ablaufen. Erst wird geladen, dann bereinigt, dann geteilt. Die Reihenfolge ist bindend: Wer vor dem Bereinigen teilt, trägt Steuerzeichen und doppelte Leerzeichen über die Chunk-Grenzen hinweg mit.

Schritt	Modul	Ausgabe
laden	pipeline/loading.py	Zahl der geladenen Dokumente
bereinigen	pipeline/cleaning.py	Bilanz aus <code>format_cleaning_stats()</code>
chunken	pipeline/chunking.py	Kennzahlen aus <code>format_chunking_stats()</code>

Strategien

Drei Splitter stehen zur Wahl. Sie unterscheiden sich darin, woran sie trennen — und damit darin, wie gleichmäßig die Stücke ausfallen.

Strategie	Trennt an	Geeignet für
recursiv e	einer Kette von Trennern, von grob nach fein	den Regelfall, auch für Handbücher
character	einem einzelnen Zeichen oder einer Zeichenfolge	fest formatierte Texte mit klarem Trenner
token	Token-Grenzen nach dem Zähler des Modells	wenn die Tokenzahl das harte Kriterium ist

Die Handbuch-Reihenfolge ist eine eigene Trenner-Kette: Sie lässt die Kapitelüberschrift vor dem Absatzumbruch greifen. Mit `--handbook` wird sie verwendet; ohne den Schalter gilt die Standardreihenfolge. Bei Handbüchern mit nummerierten Kapiteln ist die Handbuch-Reihenfolge die bessere Wahl, weil Überschriften dann nicht vom folgenden Absatz getrennt werden.

Zielgröße und Überlappung

Zwei Werte bestimmen den Zuschnitt. Die Zielgröße ist die angestrebte Zeichenzahl eines Chunks; die Überlappung ist die Zahl der Zeichen, die zwei benachbarte Chunks gemeinsam haben. Die Überlappung sorgt dafür, dass ein Satz, der genau auf eine Grenze fällt, nicht in beiden Chunks unvollständig bleibt. Ein Wert von etwa einem Achtel der Zielgröße ist ein üblicher Ausgangspunkt.

Ein Zielwert, keine Garantie

- Die Zielgröße ist ein Ziel, kein Versprechen. Ein Splitter kann einen Chunk nicht unterhalb der Größe seines Trennzeichens zerlegen. Chunks, die die Zielgröße überschreiten, werden deshalb eigens gezählt — als uebergross in den Kennzahlen.

Optionen

Option	Wirkung	Standard
paths	Quelldateien oder src:-Quellen	aus der .env
--strategy NAME	recursive, character oder token	recursive
--chunk-size N	Zielgröße in Zeichen	800
--chunk-overlap N	Überlappung in Zeichen	100
--handbook	Trenner-Reihenfolge für Handbücher	aus

Option	Wirkung	Standard
--compare	alle Strategien bei gleichem Zielwert	aus
--show N	die ersten N Chunks zeigen, 0 schaltet ab	3
--show-paths	aufgelöste Pfade vor dem Laden zeigen	aus
--progress	Fortschritt je Stufe nach stderr	aus
--write DATEI	Chunks als Textdatei speichern	—
--load-jsonl DATEI	Ladesatz speichern	—
--from-jsonl DATEI	Ladesatz statt Quellen verwenden	—

Kennzahlen

Nach dem Teilen gibt `format_chunking_stats()` den Zustand des Ergebnisses aus. Die Zahlen sind der eigentliche Zweck des Einstiegs: Sie machen sichtbar, ob der Zuschnitt zum Dokument passt.

Kennzahl	Bedeutung
Quelldokumente	Zahl der Dokumente, die in den Splitter gingen
Chunks	Zahl der entstandenen Stücke
Zeichen gesamt	Summe aller Zeichen über alle Chunks
je Chunk	min, median, mean und max der Chunk-Größen
uebergross	Chunks, die die Zielgröße überschreiten
Strategie	verwendete Strategie samt Zielgröße und Überlappung

Ein sehr großer Abstand zwischen min und max ist ein Hinweis auf einen Text, der keine gleichmäßigen Grenzen anbietet — etwa eine Tabelle oder ein langer Absatz ohne Zwischenüberschrift. Ein hoher Anteil übergroßer Chunks deutet auf dasselbe Problem hin; hier hilft eine kleinere Zielgröße oder die Handbuch-Reihenfolge mit feineren Trennpunkten.

Strategievergleich

Mit `--compare` laufen alle Strategien bei gleichem Zielwert über dieselben Dokumente. `format_strategy_comparison()` stellt die Ergebnisse nebeneinander, sodass sich Zahl,

Größenverteilung und Anteil übergroßer Chunks direkt vergleichen lassen. Der Vergleich ist die Grundlage für die Wahl der Strategie — nicht die Vermutung, welche passen könnte.

Chunk-Vorschau und Herkunft

Die Vorschau zeigt die ersten Chunks mit ihrer Herkunft. Jeder Chunk trägt in den Metadaten einen Index, die Kapitel- und Abschnittsangabe, einen Titel und seine Zeichenzahl. Damit ist im Ergebnis sichtbar, woher ein Stück stammt — auch dann, wenn die Überschrift im Text selbst keine Nummer mehr trägt.

Die Ausgabe als Markdown erhält dieselben Angaben je Chunk, sodass sie sich außerhalb des Programms prüfen lässt. Das ist besonders nützlich, bevor ein Index gebaut wird.

Ladesatz für Wiederholbarkeit

Netzbasierte Quellen sind nicht deterministisch. `--load-jsonl` legt den Ladesatz ab, bevor bereinigt und geschunkelt wird; ein späterer Lauf nimmt ihn mit `--from-jsonl` wieder auf und überspringt Laden und Netz vollständig. Beide Schalter zusammen schließen sich mit Quellenangaben aus — die Datei entscheidet dann den Inhalt, nicht die Argumente.

Exit-Codes

Code	Bedeutung
0	Lauf erfolgreich
1	Fehler: Pfad, Format oder Argumente ungültig
2	Fehlende Abhängigkeit — ein Loader-Paket ist nicht installiert